

Day - 15

Quantum Optimization & QAOA

Today we explore one of the most promising near-term applications of quantum computing: **Optimization**. We will move away from fixed algorithms and learn how quantum and classical computers can work together to solve complex problems with vast numbers of possible solutions.

1. What is Optimization?

Optimization is the process of finding the best possible solution while satisfying a set of conditions. It consists of three parts:

1. **Possible Solutions:** The search space (e.g., all possible delivery routes).
2. **Objective (Cost) Function:** A mathematical way to score a solution (e.g., total distance).
3. **Constraints:** Rules that cannot be broken (e.g., must visit every city exactly once).

The Combinatorial Explosion

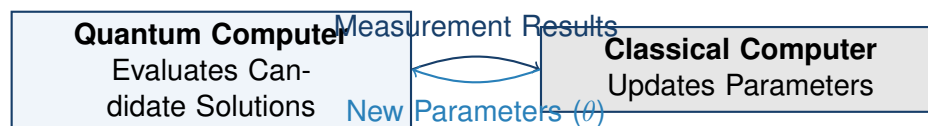
Optimization problems are hard for classical computers because the number of possible solutions grows exponentially as variables increase.

- 5 cities = 120 possible routes.
- 10 cities = ≈ 3.6 million routes.
- 20 cities = $> 2 \times 10^{18}$ routes.

Classical computers must check these one-by-one or use approximation heuristics.

2. Hybrid Quantum-Classical Optimization

Current quantum computers (NISQ era) are noisy and have limited qubits. They cannot solve these problems entirely on their own. Instead, they operate in a **Hybrid Loop** with classical processors.



3. The Optimization Loop (Step-by-Step)

1. **Prepare Parameterized Circuit:** The quantum computer creates a state $|\psi(\theta)\rangle$ where gates depend on adjustable parameters θ .
2. **Execute & Measure:** Run the circuit and collapse the state to get an outcome.
3. **Calculate Cost Function:** The classical computer evaluates how good the outcome is by calculating the **Expectation Value**: $\langle H \rangle = \langle \psi | H | \psi \rangle$.
4. **Update Parameters:** A classical optimizer (like COBYLA) suggests new parameters θ' to improve the cost score.
5. **Repeat Until Convergence:** The loop continues until the cost function stops improving, yielding the optimal parameters θ^* .

4. Quantum Approximate Optimization Algorithm (QAOA)

QAOA is a famous hybrid algorithm designed for combinatorial optimization problems like **MaxCut**. It prepares a superposition of all possible answers and then repeatedly applies two operations:

- **Problem Hamiltonian:** Alters the phase to reward "good" solutions (constructive interference).
- **Mixer Hamiltonian:** Explores new solutions to prevent getting stuck in local minimums.

5. Qiskit Implementation (QAOA for MaxCut)

This code demonstrates setting up a graph, converting it to an optimization problem, and running the QAOA hybrid loop using Qiskit.

```
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler
from qiskit_optimization.applications import Maxcut
import networkx as nx

# 1. Define the Problem (A 4-node Graph)
graph = nx.Graph()
graph.add_edges_from([(0,1), (1,2), (2,3), (3,0)])

# 2. Convert to Quantum-Friendly Format (Ising Model)
maxcut = Maxcut(graph)
operator, offset = maxcut.to_quadratic_program().to_ising()

# 3. Setup Hybrid Loop Components
optimizer = COBYLA(maxiter=100) # Classical Optimizer
sampler = StatevectorSampler() # Quantum Execution Engine

# 4. Initialize and Run QAOA
qaoa = QAOA(sampler=sampler, optimizer=optimizer, reps=2)
result = qaoa.compute_minimum_eigenvalue(operator)

print("Optimized_Result:", result)
```

6. Deconstructing Common Misconceptions

Myth	Reality
"Quantum optimization checks every possible solution simultaneously."	Quantum algorithms manipulate probabilities. Better solutions undergo constructive interference, making them more likely to be measured.
"Quantum computers will replace classical computers for these problems."	Today's algorithms are hybrid. The quantum hardware only evaluates the cost space; the classical hardware drives the parameter optimization.
"QAOA always gives the absolute perfect solution."	QAOA is an <i>approximate</i> algorithm. The goal is to find a very good, near-optimal solution highly efficiently.